

Competitive Programming

Section 32 Editorial: Combinatorics Under Modulo

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	Shared tool: factorials under a prime modulus	4
3	A. Choose Queries	5
3.1	Restatement and constraints	5
3.2	Observations and walk-through	5
3.3	Pseudocode	5
3.4	Correctness proof	5
3.5	Complexity	5
3.6	Implementation notes and wrong answers	6
3.7	Full C++ solution	6
3.8	Full Python solution	6
4	B. Rearrange Letters	8
4.1	Restatement and constraints	8
4.2	Observations and walk-through	8
4.3	Pseudocode	8
4.4	Correctness proof	8
4.5	Complexity	8
4.6	Implementation notes and wrong answers	9
4.7	Full C++ solution	9
4.8	Full Python solution	9
5	C. Distribute Candies	11
5.1	Restatement and constraints	11
5.2	Observations and walk-through	11
5.3	Pseudocode	11
5.4	Correctness proof	11
5.5	Complexity	11
5.6	Implementation notes and wrong answers	11
5.7	Full C++ solution	12
5.8	Full Python solution	12
6	Testing and stress discussion	14
7	What to carry forward	15

1 Editorial overview

Problem	Mathematical object	Required factorial limit
A. Choose Queries	$C(n, k)$	maximum n
B. Rearrange Letters	$n! / \text{product}(c!)$	string length
C. Distribute Candies	$C(n+s-1, n-1)$	maximum $n+s-1$

2 Shared tool: factorials under a prime modulus

All local factorial indices are below $\text{MOD}=1,000,000,007$; this is a required precondition, not a cosmetic fact. It ensures $i!$ has an inverse modulo MOD . The local maximum index is at most two million, so the technique is valid.

Store arrays indexed from 0 through `limit` with the invariant:

```
fact[i]      = i! mod MOD
invfact[i]   = inverse(i!) mod MOD
```

```
fact[0] = 1
for i = 1..limit: fact[i] = fact[i-1] * i
invfact[limit] = fact[limit]^(MOD-2)
for i = limit down to 1: invfact[i-1] = invfact[i] * i
C(n,k) = 0 if k < 0 or k > n, otherwise fact[n]*invfact[k]*invfact[n-k]
```

The backward recurrence follows from $i! = i \cdot (i-1)!$. It costs $O(\text{limit})$ time, one $O(\log \text{MOD})$ exponentiation, and $O(\text{limit})$ memory; $C(n,k)$ is then $O(1)$. In C++, multiply as `long long`; in Python use `pow(x, MOD-2, MOD)`.

3 A. Choose Queries

3.1 Restatement and constraints

Answer up to 200,000 values of $C(n,k)$ for $0 \leq k \leq n \leq 10^6$. Computing the product from scratch per query is too slow, and ordinary division after a modulo is invalid. Read all queries first because the largest n determines the one shared preprocessing limit.

3.2 Observations and walk-through

For $C(6,2)$, ordinary arithmetic says $6!/(2!4!)=15$. The code instead multiplies three residues:

```
fact[6] * invfact[2] * invfact[4] mod MOD = 15
```

For $C(10,0)$ and $C(10,10)$, the same formula uses $\text{fact}[0]=\text{invfact}[0]=1$, so both answers are one without branches.

3.3 Pseudocode

```
read all (n,k) queries
prepare(maximum n)
for each (n,k): print C(n,k)
```

3.4 Correctness proof

The shared invariant gives $\text{fact}[n]=n!$ and inverse factorials for both denominator terms. Since all indices are below prime MOD , those denominator factorials are invertible. Their product with $\text{fact}[n]$ is exactly $n!/(k!(n-k)!)$ in modular arithmetic, which is $C(n,k)$. $\square$

3.5 Complexity

```
Preprocessing:  $O(\max n)$  time and memory
Each query:     $O(1)$  time
```

3.6 Implementation notes and wrong answers

Do not precompute `invfact[i]` with an exponentiation inside a loop. Do not omit `fact[0]=1`. A generic combination helper should reject invalid `k` even though this local input guarantees a valid range.

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r * a % MOD; a = a * a % MOD; e >>= 1; }
    return r;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; if (!(cin >> q)) return 0;
    vector<pair<int,int>> queries(q); int limit = 0;
    for (auto &[n,k] : queries) { cin >> n >> k; limit = max(limit, n); }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[limit] = power(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    for (auto [n,k] : queries)
        cout << fact[n] * invfact[k] % MOD * invfact[n-k] % MOD << '\n';
}
```

3.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact
```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
    fact, invfact = prepare(max(n for n, _ in queries))
    print("\n".join(str(fact[n] * invfact[k] % MOD * invfact[n-k] % MOD)
                     for n, k in queries))

if __name__ == "__main__":
    main()

```

4 B. Rearrange Letters

4.1 Restatement and constraints

Given one lowercase string of length at most 10^6 , count its distinct rearrangements modulo MOD. The trap is that equal letters are not distinct objects, so a direct $n!$ overcounts.

4.2 Observations and walk-through

For aabc, pretend the two as are labelled a_1, a_2 . There are $4!=24$ labelled permutations, but a_1abc and a_2abc are the same visible string. Every visible arrangement is counted $2!$ times, so the answer is $24/2=12$. For aabbc, two independent duplicate groups give $5!/(2!*2!)=30$.

4.3 Pseudocode

```
count every letter
prepare(length of string)
answer = fact[length]
for each letter count c:
    answer = answer * invfact[c] mod MOD
print answer
```

4.4 Correctness proof

Label all equal copies temporarily. There are $n!$ labelled orders. For a fixed visible arrangement, the copies of each character can be permuted in $c!$ ways without changing it; independent character groups multiply these counts. Therefore each visible arrangement has exactly $\text{product}(c!)$ labelled preimages, and multiplying by inverse factorials computes the quotient. $\square$

4.5 Complexity

```
Time:     $O(n + \text{alphabet size})$ 
Memory:  $O(n)$ 
```


4.6 Implementation notes and wrong answers

Use a 26-entry frequency array; counts of absent characters are zero and `invfact[0]=1`, so they are harmless. Recomputing `inverse(fact[c])` separately for every letter is acceptable for 26 letters but hides the reusable engine. Do not count $n!$ when the string has duplicates.

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;
long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r*a % MOD; a = a*a % MOD; e >>= 1; }
    return r;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    string s; if (!(cin >> s)) return 0;
    int n = (int)s.size();
    vector<long long> fact(n + 1, 1), invfact(n + 1, 1);
    for (int i = 1; i <= n; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[n] = power(fact[n], MOD - 2);
    for (int i = n; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    int count[26]{};
    for (char ch : s) ++count[ch - 'a'];
    long long answer = fact[n];
    for (int c : count) answer = answer * invfact[c] % MOD;
    cout << answer << '\n';
}
```

4.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def main() -> None:
    s = sys.stdin.buffer.read().strip()
    if not s:
        return
    n = len(s)
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
```

```

invfact = [1] * (n + 1)
invfact[n] = pow(fact[n], MOD - 2, MOD)
for i in range(n, 0, -1):
    invfact[i - 1] = invfact[i] * i % MOD
counts = [0] * 26
for ch in s:
    counts[ch - ord('a')] += 1
answer = fact[n]
for c in counts:
    answer = answer * invfact[c] % MOD
print(answer)

if __name__ == "__main__":
    main()

```

5 C. Distribute Candies

5.1 Restatement and constraints

For each of up to 200,000 queries, distribute s identical candies among n distinct children, allowing zero. Both are at most 10^6 , so the largest factorial index is $n+s-1$, at most 1,999,999, still strictly below MOD. Precomputing merely to maximum n is a frequent wrong answer.

5.2 Observations and walk-through

The equation is $x_1 + \dots + x_n = s$ with nonnegative variables. For $n=3, s=4$, the string `**|**` represents (2,1,1). There are four stars and two bars; choose two bar positions among six, giving $C(6,2)=15$. For $n=1, s=10$, choose zero bars, $C(10,0)=1$.

5.3 Pseudocode

```
read all (n,s) queries
prepare(maximum over queries of n+s-1)
for each query: print C(n+s-1, n-1)
```

5.4 Correctness proof

Map a distribution to a row containing x_1 stars, a bar, x_2 stars, and so on. This mapping is reversible: the stars in each gap recover the unique distribution. There are $s+n-1$ positions and selecting the $n-1$ bar positions therefore counts all and only valid distributions. `\square`

5.5 Complexity

```
Preprocessing: O(L) time and memory, L = max(n+s-1)
Each query:    O(1) time
```

5.6 Implementation notes and wrong answers

Read all queries before allocating so the maximum uses $n+s-1$. Children are distinct but candies are identical: use bars, not a permutation count. The formula already handles $s=0$ and $n=1$; extra branches are unnecessary.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;
long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r*a % MOD; a = a*a % MOD; e >>= 1; }
    return r;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; if (!(cin >> q)) return 0;
    vector<pair<int,int>> queries(q); int limit = 0;
    for (auto &[n,s] : queries) { cin >> n >> s; limit = max(limit, n+s-1); }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[limit] = power(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    for (auto [n,s] : queries) {
        int all = n + s - 1, bars = n - 1;
        cout << fact[all] * invfact[bars] % MOD * invfact[all-bars] % MOD << '\n';
    }
}
```

5.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
```

```

queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
fact, invfact = prepare(max(n + s - 1 for n, s in queries))
out = []
for n, s in queries:
    all_positions = n + s - 1
    bars = n - 1
    out.append(str(fact[all_positions] * invfact[bars] % MOD
                  * invfact[all_positions - bars] % MOD))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

6 Testing and stress discussion

For A, compare small values with `math.comb`. For B, enumerate permutations of short strings and compare the number of unique strings. For C, enumerate small nonnegative vectors recursively, then compare with the formula. Important fixed tests are:

```
A: k=0, k=n, and the largest n
B: all equal letters and all distinct letters
C: s=0, n=1, and a query that determines the global maximum n+s-1
```

Run the supplied checks with:

```
CP_TARGET=solution python3 sections/32_combinatorics_under_modulo/check.py
```

7 What to carry forward

First translate a story into a mathematical object. Then determine the largest factorial index *before* coding. Prime-modulus factorial preprocessing is a powerful constant-time query engine, but only while every relevant index remains strictly below the modulus.